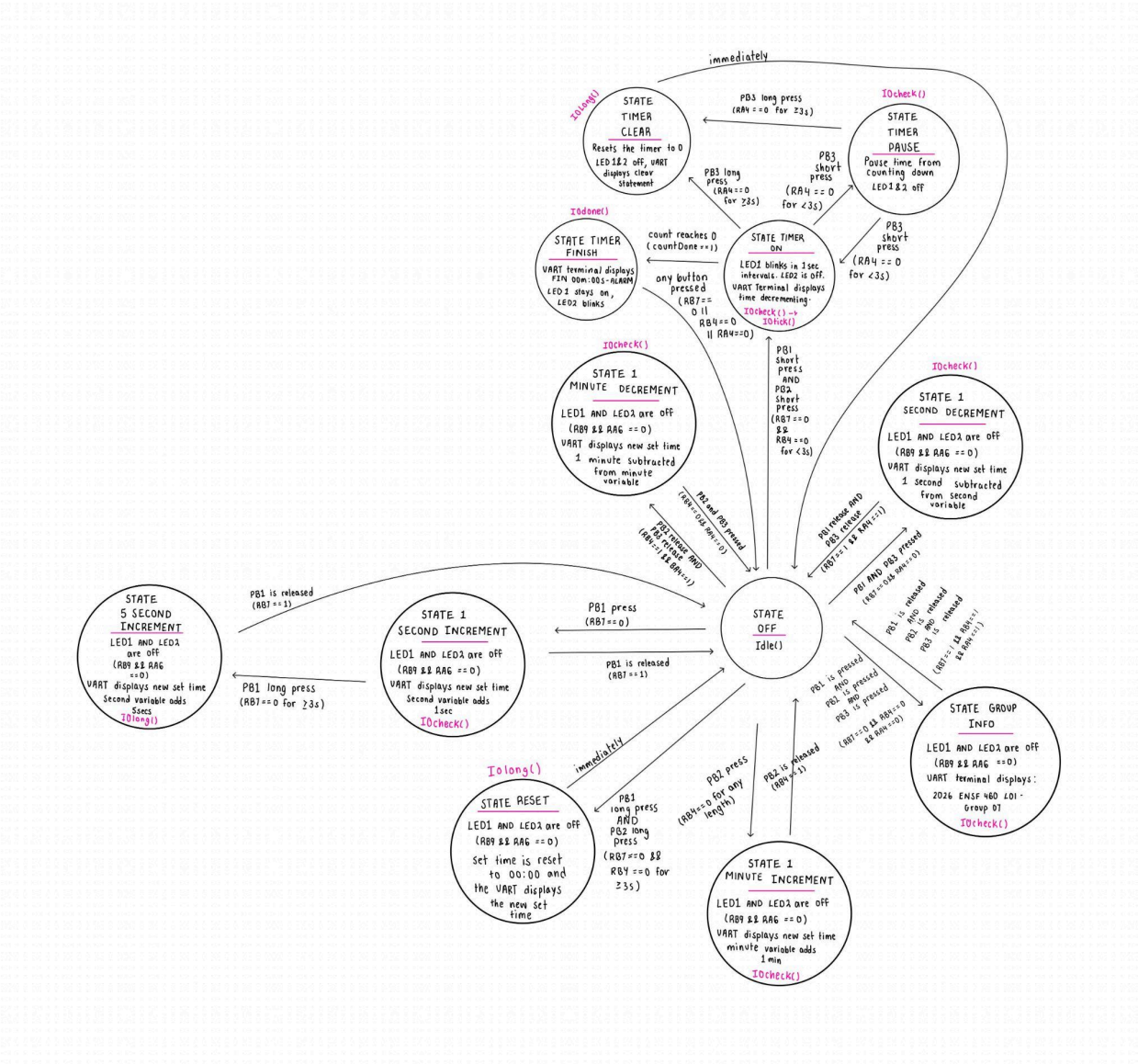
ENSF 460 – Embedded Software and Hardware Systems

Project 1 - Countdown Timer

Group 7

Basema Khan (30204130)
Michelle Yoon (30189382)
Vincent Fong (30170587)

State Diagram



Notes:

1. *The function beside each state is the function called from main() that enters or runs that state.*
2. *“Immediately” in the state diagram means the transition happens automatically right after the state’s actions are performed, with no button press or other event needed. TIMER CLEAR and RESET are shown as separate states for clarity, but in the code they’re actions performed inside IOlong() before returning to state OFF.*

Code Implementation

Main.c

The main() function in Main.c starts by switching the clock to 500 kHz, and then calls the initialization functions: InitUART2(), IOinit() and timer1_init(), which initialize the UART, inputs/outputs and Timer1 respectively. Then, IOstart() is called, which prints “SET 00m : 00s” on the UART terminal and begins the system in the SET state.

Then, the main loop, while(1), is entered. In this loop, we start off by silencing the UART interrupts so that they don’t wake the CPU and be mistaken for a Timer1 tick.

```
IEC1bits.U2TXIE = 0;
IFS1bits.U2TXIF = 0;
```

Disabling these interrupts and clearing the flag at the top of every loop keeps the program in Idle() until a real event happens.

An if-statement is used to determine if any interrupt-generated events are waiting to be processed, ensuring the CPU only enters low-power Idle mode if no event is waiting. The events monitored are cnFlag, longPressButton, countDone, alarmTick, comboDone and normalTick.

The rest of main() is event handling. Each block follows the same pattern: an ISR sets a flag, main clears it and calls the handler function.

- countDone: if countDone is set to 1, Timer2/3 has reached the set time so the countdown has finished and IOdone() is called, which clears the flag and enters the TIMER FINISH state (prints FIN 00m : 00s - ALARM, turns LED1 on and starts LED2 blinking)
- cnFlag: If cnFlag is set to 1, a button was pressed or released while the timer is running. The flag is cleared and IOcheck() is called to handle pausing, resuming or exiting the countdown.
- comboDone: If comboDone is set to 1, the 100 ms wait started by the CN interrupt has ended. This wait gives the user time to press several buttons together so they’re read as one combination. The flag is cleared and IOcheck() is called to handle the actions (incrementing or decrementing the time, showing the group info or starting the countdown).
- longPressButton: If longPressButton is set to 1, a button has been held for 3 seconds, defined as a long press. The flag is cleared and IOlong() is called to handle the long-press scenario (switching to 5-second increments, resetting the time or clearing the timer).
- alarmTick: if alarmTick is set to 1, Timer1 is in alarm mode and about 0.25 seconds have passed. The flag is cleared and IOtick() is called to blink LED2 (TIMER FINISH state).
- normalTick: If normalTick is set to 1, one second has passed in Timer1. The flag is cleared and IOtick() is called.

It’s very important to clear the flags upon every event because otherwise, the event will repeat and the CPU never enters Idle.

```
// Handle countdown completion signaled by the Timer 3 ISR
if (countDone) {
    IOdone();
}

// Process button changes signaled by the CN interrupt
if (cnFlag) {
    cnFlag = 0;         // Clear the pending button change flag
    IOcheck();          // Read buttons and handle the correct action
}

// Process SET mode button combinations after the ~100ms detection window
if(comboDone){
    comboDone = 0;      // Clear the completed combination event
    IOcheck();          // Read the final button combination
}

// Handle a 3-second button hold detected by Timer 1
if (longPressButton) {
    longPressButton = 0;        // Clear the long-press event flag
    IOlong();                   // Execute the state-dependent long-press action
}

// Handle ~0.25 second alarm ticks by updating the FIN state LED
if(alarmTick){
    alarmTick = 0;          // Clear the pending alarm tick flag
    IOtick();               // Toggle LED2 while in FIN mode
}

// Handle normal 1 second Timer 1 events
// Updates held-button adjustments or the running countdown display
if (normalTick) {
    normalTick = 0;     // Clear the pending one second tick
    IOtick();           // Perform the state-dependent periodic update
}
```

The structure of the event handling.

TimeDelay.c

The TimeDelay.c file contains the timer initialization code for Timer1 and the combined Timer2 and Timer3, as well as functions that perform pausing, resuming, and returning the state of the countdown timer and differentiating between long and short button presses.

A clock oscillation of 500 kHz and a prescaler of 1:8 was chosen for all Timers.

Constants and Variables:
TICKS_PER_SEC = 31250: there are 31250 ticks per seconds, and this value is defined so as to easily be used in calculations to convert seconds to ticks, which can then be assigned to PR2 and PR3 registers.
static volatile uint8_t seconds: Timer1 counted seconds. Begins counting upon a CN interrupt, is incremented by 1 upon every _T1Interrupt, and upon reaching 3, defines a press to be long.
static volatile uint8_t timedButton = 0: button currently being timed.
static uint32_t countPeriod = 0: total ticks loaded into PR2 and PR3.
volatile uint8_t longPressButton = 0: button which is long pressed.
volatile uint8_t countDone = 0: 0 initially, switched to 1 when the timer reaches the PR value, indicating the countdown is finished.
static volatile uint8_t alarmMode: Indicates whether TImer1 is operating in alarm mode.
volatile uint8_t alarmTick: Signals that an alarm tick has occurred.
static volatile uint8_t comboMode: Indicates whether Timer 1 is detecting button combinations.
volatile uint8_t comboDone: Signals that the combination-detection interval has finished.

Volatile uint8_t normalTick: Signals that a normal one-second Timer1 interrupt has occurred.

We decided to use CN interrupts combined with Timer1 to identify short versus long presses. When a CN interrupt detects a button change during SET mode, comboTimer_start() first creates a 100ms window to recognize button combinations so there is no issue with a button pressing slightly before another. After this interval, IOcheck() processes the buttons and calls pressTimer_start(button), shown in figure 1 to begin tracking the press duration. While held, Timer1 interrupts every second, and _T1Interrupt adds 1 to the seconds variable, as shown in Figure 2. If the seconds variable reaches 3 seconds, Timer1 is stopped and longPressButton is set to whichever button was pressed. This is so Timer1 doesn't unnecessarily continue counting when the condition for a long button press has already been met. When the button is released, the CN interrupt calls the pressTimer_stop() function, which returns whether Timer1 is still running by using the value of T1CONbits.TON, which would be 1 if running and 0 if not, as shown in Figure 3. If it is running (returns 1), a short press is identified, and if not (returns 0), it means a long press has already been identified. Timer1 is also reconfigured for 100ms combination detention and 0.25 second alarm ticks using comboMode and alarmMode to distinguish between its different operations. The 100ms is to help with not having 2 inputs accidentally when double pressing. The 0.25 second alarm timer is to help LED2 blink when the alarm is finished.

To recognize button combinations during SET mode, the program uses Timer1 to create a ~100ms detection window. When a CN interrupt detects a button change, _CNInterrupt() calls comboTimer_start() (figure 7), which stops and resets Timer1, sets comboMode to 1, and configures PR1 = 3125. Once the 100ms interval has elapsed, _T1Interrupt() stops Timer1, clears comboMode, and sets comboDone to 1 (figure 2). The main loop then detects this flag and calls IOcheck() to read the current button combination. This allows buttons pressed close together to be processed as a single combination rather than as separate inputs.

```
// Start Timer 1 with 1 seconds ticks
// A nonzero button argument enables 3-second long press detection
// PAssing 0 produces normal 1 second ticks without tracking button press duration
void pressTimer_start(uint8_t button) {
    T1CONbits.TON = 0;      // Turn off Timer 1

    // Disable Timer 1's previous alarm and combination detection modes
    alarmMode = 0;
    alarmTick = 0;
    comboMode = 0;
    comboDone = 0;

    PR1 = 31250;            // Restore the 1 second Timer 1 period
    timedButton = button;   // Track the button, 0 disables long-press detection
    seconds = 0;            // Reset the number of elapsed 1 second ticks
    TMR1 = 0;               // Reset Timer 1
    IFS0bits.T1IF = 0;      // Clear the Timer 1 interrupt flag
    T1CONbits.TON = 1;      // Start Timer 1 with new settings
}
```

Figure 1: The pressTimer_start(button) function, which configures Timer1 for one second intervals and button hold detection.

```c
void __attribute__((interrupt, no_auto_psv)) _T1Interrupt(void) {
    IFS0bits.T1IF = 0;              // Clear Timer 1 interrupt flag

    //Combination mode: The ~100ms button detection window has ended
    if(comboMode){
        T1CONbits.TON = 0;          // Stop the single run combination Timer
        comboMode = 0;              // Exit combination mode
        comboDone = 1;              // Signal to main to process the button combination
        return;                     // Return to not process it like a normal Timer 1 one second tick
    }

    // Alarm mode: Generate a repeated ~0.25 second event for LED2 blinking
    if(alarmMode){
        alarmTick = 1;              // Signal to main to toggle LED2 through IOtick()
        return;                     // Return to not process it like a normal Timer 1 one second tick
    }

    seconds++;                      // Count how many one second Timer 1 interrupts occur
    normalTick = 1;                 // Signal to main to perform a a normal periodic update

    // Detect a 3-second hold only when a button is being tracked
    if (seconds >= 3 && timedButton != 0) {
        T1CONbits.TON = 0;                  // Stop Timer 1 after detecting a 3 second hold
        longPressButton = timedButton;      // Signal to main that a long press occurred
    }
}
```

Figure 2: _T1Interrupt, which handles button-combination detection, alarm timing, one-second updates, and long-press detection

```c
// Called from the CN ISR when the button is released.
// Returns 1 if still running (short press), 0 if long press already reported.
uint8_t pressTimer_stop(void) {
    uint8_t wasRunning = T1CONbits.TON;
    T1CONbits.TON = 0;
    return wasRunning;
}
```

Figure 3: pressTimer_stop(), which is called when a button is released.

Timer2 and Timer3 are combined to form a 32-bit register (T2CONbits.T32 = 1) that counts upwards toward the programmed countdown duration We decided to combine the registers because in order to fulfill the maximum possible time condition (59:59), 16-bits would be too small. The countTimer_start(totalSeconds) function takes in a time argument, converts it to ticks and sets PR3 to the upper 16-bits and PR2 to the lower 16-bits of this number, as shown in Figure 4. It then starts counting up, stopping when the PR value is reached. Figure 5 shows the function countTimer_pause(), which can be used to pause the countdown timer at its current value, and Figure 6 shows the function countTimer_resume(), which resumes the timer from where it was paused. The function countTimer_remaining() returns the seconds left and is used by IOs.c to print the time as it decrements. When the count reaches the PR value, T3Interrupt fires and sets countDone to 1, which the IO code can then use to ensure the finish statement is correctly shown.

- countPeriod = (uint32_t)totalSeconds * TICKS_PER_SEC: formula used to convert a time argument into ticks which can be used in PR2 and PR3.

When the countdown reaches zero, the Timer3 interrupt sets countDone to 1. The main loop calls IOdone(), which changes the application state to FIN and calls alarmTimer_start(). The alarmTimer_start() (figure 8) function stops and resets Timer1, sets alarmMode to 1, and configures PR1 = 7812 to generate interrupts every ~0.25 seconds. During alarm mode,

_T1Interrupt() sets alarmTick to 1 (figure 2). The main loop detects this flag and calls IOtick(), which toggles LED2 while the application is in FIN mode. This produces a repeating alarm indication until a new button press dismisses the alarm and returns the program to SET mode.

The countTimer_remaining() function (figure 9) calculates the remaining countdown time using the current value of the combined 32-bit Timer 2/3 counter. First the lower 16 bits are read from TMR2, followed by the upper 16 bits from TMR3HLD. These values are combined using a left shift of 16 bits and an OR operation to reconstruct the elapsed timer count. The elapsed ticks are subtracted from countPeriod to calculate the remaining timer ticks. The result is divided by TICKS_PER_SEC to convert the remaining ticks into seconds. The function is called during RUN mode to update the UART countdown display and when resuming from PAUSE mode to display the correct time.

```
// Load the set time and start counting from zero
void countTimer_start(uint16_t totalSeconds) {
    if (totalSeconds == 0) {
        return;                    // nothing to count
    }

    countPeriod = (uint32_t)totalSeconds * TICKS_PER_SEC;

    T2CONbits.TON = 0;
    T2CONbits.T32 = 1;           // combine Timer2 + Timer3
    T2CONbits.TCS = 0;           // Fcy
    T2CONbits.TCKPS = 1;           // 1:8

    TMR3HLD = 0;                   // high word first
    TMR2    = 0;                   // then low word
    PR3 = (uint16_t)(countPeriod >> 16);
    PR2 = (uint16_t)countPeriod;

    IPC2bits.T3IP = 3;
    IFS0bits.T3IF = 0;
    IEC0bits.T3IE = 1;

    countDone = 0;
    T2CONbits.TON = 1;
}
```

Figure 4: countTimer_start(seconds), which takes in a time in seconds and begins counting to it.

```
// Stop counting but keep the current count
void countTimer_pause(void) {
    T2CONbits.TON = 0;
}
```

Figure 5: countTimer_pause(), which pauses the timer.

```
// Continue from where it was paused
void countTimer_resume(void) {
    if (!countDone && countPeriod != 0) {
        T2CONbits.TON = 1;
    }
}
```

Figure 6: countTimer_resume(), which resumes the timer from where it was paused.

```
// Start a ~100ms window to recognize closely spaced button presses
void comboTimer_start(void){
    T1CONbits.TON = 0;          // Stop Timer 1 before reconfiguring it

    comboMode = 1;              // Tell Timer 1 ISR that combination detection is active
    comboDone = 0;              // Clear previous combination complete flag

    TMR1 = 0;                   // Reset Timer 1
    PR1 = 3125;                 // Set ~100ms period
    IFS0bits.T1IF = 0;          // Clear any pending Timer 1 interrupt
    T1CONbits.TON = 1;          // Start the ~100ms combination timer
}
```

Figure 7: comboTimer_start() which creates a 100ms delay to detect combo presses.

```
// Reconfigure Timer 1 for ~0.25 second alarm ticks
// Signal to main to toggle LED2 while in FIN mode
void alarmTimer_start(void){
    T1CONbits.TON = 0;          // Stop Timer 1 before reconfiguration

    alarmMode = 1;              // Enable alarm mode in the Timer 1 ISR
    alarmTick = 0;              // Clear pending alarm events

    seconds = 0;                // Reset Timer 1's elapsed seconds counter
    timedButton = 0;            // Disable long-press tracking

    TMR1 = 0;                   // Reset Timer 1 counter
    PR1 = 7812;                 // Set ~0.25 second timer period

    IFS0bits.T1IF = 0;          // Clear pending Timer 1 interrupt flag
    T1CONbits.TON = 1;          // Start Timer 1 in alarm mode
}
```

Figure 8: alarmTimer_start() which creates a 0.25 second delay for LED2 blinking.

```
// Seconds left, rounded up so it shows 1 until the time actually ends
uint16_t countTimer_remaining(void) {

    // Returns zero if the countdown is complete or no period is configured
    if (countDone || countPeriod == 0) {
        return 0;
    }

    uint16_t lo = TMR2;             // Read the lower 16-bits of Timer 2/3 current count
    uint16_t hi = TMR3HLD;          // Read the upper 16-bits of Timer 2/3 current count
    uint32_t elapsed = ((uint32_t)hi << 16) | lo;       // Reconstruct the 32-bit elapsed tick count from both Timer registers

    // Return zero if the elapsed count has reached the countdown period
    if (elapsed >= countPeriod) {
        return 0;
    }

    uint32_t left = countPeriod - elapsed;      // Subtract elapsed ticks from the total ticks to get remaining ticks
    return (uint16_t)((left + TICKS_PER_SEC - 1) / TICKS_PER_SEC);      // Convert remaining ticks to seconds
}
```

Figure 9: countTimer_remaining() which calculates the remaining seconds using 32-bit Timer2/3 counter.

IOs.c

Constants and variables

Each button is given its own bit, so any combination of buttons fit in a single uint8_t. Instead of using the register name repeatedly, we defined PB1, PB2, PB3, START and ALL variables for when each button is pressed or when two of them and all are pressed. Enum {SET, RUN, PAUSE, FIN} were used for four states: when timer is set, timer run, timer pause and timer finish. The CN flag is set by the interrupt and read by the main loop. Static variables such as state, buttons, held, minutes, seconds, wasLong, and fast are all initialized and these will be used below to interact with button presses.

For this assignment, we use bit number: read PORTB and PORTA once, right after each CN interrupt and save the result. Repeatedly testing PORTA and PORTB is also a type of polling since the while loop from main calls IOcheck() and waits for the buttons to be pressed every time. CPU should be in idle and only look at buttons when CN interrupt says something changed. So, by using this method, right after CN interrupt code works with the saved value.

```
#define PB1 1
#define PB2 2
#define PB3 4
#define START (PB1 | PB2)
#define ALL   (PB1 | PB2 | PB3)

enum { SET, RUN, PAUSE, FIN };
```

Figure 10: How we set up

IOstart()

This is where the Uart terminal first displays the timer. It is set to be SET 00m : 00s. \r moves the cursor to the start of the line without a new line. This way it will overwrite the previous one. minutes/10 gives the ten digits and minutes % 10 gives the ones digit. Adding ‘0’ just converts a number from 0 to 9 into ASCII character. 1 is for sending the character once.

```
XmitUART2('0' + minutes / 10, 1);
XmitUART2('0' + minutes % 10, 1);
Disp2String("m : ");
XmitUART2('0' + seconds / 10, 1);
XmitUART2('0' + seconds % 10, 1);
Disp2String("s               ");
```

Figure 9: Uart display

Setting the time: step()

This function changes time (minutes and seconds) by pressing buttons. It either increments, decrements. With PB1 alone, seconds increase. Normally it adds 1; seconds = (seconds +1) % 60. But if held more than 3 seconds, it increments by 5; if (fast && seconds % 5 ==0) seconds = (seconds +5) % 60; For PB2 'minutes' variable is used instead and it works the same as 'seconds' for PB1. It also increments by 1 each time PB2 is pressed. However, for the decrements, we used ? operator to check if the seconds is 0, set to 59; otherwise set it to seconds -1. Making the integers wrap from 59 to 0 when incrementing or 0 to 59 when decrementing was very crucial and both ? 59 and %60 were used to have this feature.

```
if (held == PB1) {
    if (fast && seconds % 5 == 0) seconds = (seconds + 5) % 60;
    else                          seconds = (seconds + 1) % 60;
} else if (held == PB2) {
    minutes = (minutes + 1) % 60;
} else if (held == (PB3 | PB1)) {
    seconds = (seconds == 0) ? 59 : seconds - 1;
} else if (held == (PB3 | PB2)) {
    minutes = (minutes == 0) ? 59 : minutes - 1;
} else {
    return;              // PB3 alone or other combos: no change
```

Figure 10: Shows how the timer minutes and seconds increment/decrement is calculated

startCount(): starting the countdown

To set up a count down start state, we converted the total time to total seconds and saved it to uint_16 total variable. If total=0 it should return, nothing will count and the terminal will still print SET. For the actual countdown we called countTimer_start() with the parameter 'total' and pressTimer_start(0); from TimeDelay.c. Now, the state is set to RUN. countTimer_start() runs by timer 2 and 3 combined and it fires once after the whole duration for actually counting down. Meanwhile, pressTimer_start() runs by Timer 1 with 1 second ticks to update the display. Now the terminal will print CNT instead.

IOcheck(): the main button handler

'now' shows the current button value, while 'added' holds the buttons that are down now but not before. In simple words, it detects which button was newly pressed. So flipping 'buttons'

flips the old bits, and ANDing with now keeps only newly pressed ones. For example if buttons is PB1(001) and now is PB1+PB2(011) then ~buttons & now = 010 meaning that button 2 (010) just went down. If now == buttons, that means the same button has been pressed the whole time and nothing changed.

```
uint8_t now = readButtons();
uint8_t added = now & ~buttons;   // buttons that just went down
```

Figure 11: shows how CN interrupt flags the changes

While state is PAUSE or RUN, and now ==0 meaning currently, no button is pressed, the timer should stop. We call pressTimer_stop() from TimeDelay.c. Within this if statement, if the state is RUN and PB3 is newly pressed for a short period of time, timer count should pause. This is also called from TimeDelay.c as counterTimer_pause() function. Timer 1 has already stopped by pressTimer_stop() function and the state is now set to PAUSE so the display freezes. Else, if the PB3 is pressed for a short time but state is PAUSE, we resume the timer (countTimer_resume()) and start the timer by pressTImer_start(0). In this case we have to check the remaining time from the timer in order to display it on our terminal. We 'left' variable that calls the countTimer_remaining() function. 'Left' is the subtraction of elapsed time from the total count period. If PB3 is just pressed alone, nothing happens but the countdown is still on.

When every button is released, if PB1 and PB2 were both pressed for short period(held==START), we will start the count and if all three buttons were pressed, group info should be showing. So we overwrite that info with SET minutes:seconds. These minutes and seconds should be the same value from where the timer was paused due to pressing all the buttons. Note that everything starts on release of the button and not on press, the display has to go back to the regular timer from group info.

If state is FIN, and alarm message has gone up, we have to clear this display and go back to the regular timer SET. then we follow the same rule, change state to SET and display SET minutes:seconds. However, if now==ALL, this means currently all three buttons are being pressed. So the group info should be on display.

IOlong():

IOlong() runs everytime Timer1 has been running for 3seconds and switches itself off. This is to differentiate the time we press the button. If the button is pressed less than 3 seconds, we consider it as a short press and if longer than 3, it is a long press. Each action has different results. There could be three cases. First, if PB3 is held for three seconds during RUN or PAUSE mode, the countdown is cleared, the time is reset to 00m : 00s and it returns to SET mode. If PB1 and PB2 are held for long time, timer set will be cleared and it will display 00m:00s. For both cases we are going to set minutes and seconds variable to 0. Lastly, if PB1 is held for a long time the increments will happen fast.

IOtick()

IOtick() runs once per second while TImer1 is running. If state is SET, it means a button is still held so the program needs to jump to step() and do its action. If state is RUN, Timer 1 generates one second events that cause IOtick() to toggle LED1 and retrieve the remaining countdown time using countTimer_remaining(). The remaining seconds are converted into minutes and seconds, which are then displayed on the UART terminal. If the remaining time reaches zero, the function returns without updating the display, since the countdown completion is handled by IOdone(). During FIN mode, IOtick() is called approximately every 0.25 seconds and toggles LED2 to indicate that the countdown has finished.

IOdone()

We start off with countDone=0; to lower the flag that Timer2/3 ISR raised. The main loop calls IOdone() when the countDone= 1. So to prevent IOdone() looping, we have to clear. If (state != RUN) return; is a safety check to make sure alarm only happens if the timer is actually counting. pressTimer_stop() stops the timer since it is no longer needed. State is set to FIN and minutes and seconds are all set to 0. LED1 is turned on while LED2 is initially turned off. The alarmTimer_start() function then configures Timer1 to generate approximately 0.25 second interrupts, allowing LED2 to blink, while the UART terminal shows FIN 00m : 00s - ALARM.

Corner Cases

What happens if the user tries to start the timer with a time 00:00?

: Before starting the timer, the program calculates the total time set up by converting the time into total seconds and check if that time is 0 or not. If it is 0, which will be the case for trying to start the timer when it is SET 00:00, nothing happens and just returns.

```
static void startCount(void) {
    uint16_t total = minutes * 60 + seconds;    // Convert minut

    // Do not start a countdown if the selected time is zero
    if (total == 0) {
        return;
```

Figure 12: How the program checks if the time is 00:00